

# 独ソ戦 3D Historical Visualizer

## GitHub Cloud 開発基盤 基礎設計 v0.3

**Status:** Reviewed Baseline Proposal

**Date:** 2026-08-12

**Architecture / Independent Review:** GPT SOL

**Upstream Foundation:** Project Foundation v0.1

**Additional Delivery Constraint:** Desktop + Smartphone Browser Support

---

## 0. 結論

本プロジェクトの GitHub 開発基盤は、以下を基礎構成とする。

- **GitHub Free** を前提とする。
  - Repository は **GitHub Free Organization 配下の Public Single Repository** を推奨する。
  - main は **Protected Branch** とし、Pull Request と Required Status Checks を必須化する。
  - main への push / merge 権限は、可能な限り **Human Project Owner のみ** に限定する。
  - 開発 branch は短命 branch とし、develop 等の長期統合 branch は設けない。
  - Merge 方式は **Squash Merge のみ** とする。
  - **Codex が実装・テスト・修正**を担当し、人間 Project Owner はコーディング・デバッグを行わない。
  - Build / Test / Validation の正本は **GitHub Actions** とする。
  - **GitHub Codespaces は初期構成に含めない**。
  - GitHub Pages は Production 公開候補とするが、Technology Selection 後に正式決定する。
  - Historical Data は Application code と同一 Repository に配置するが、専用 namespace・専用 Validation を持つ設計とする。
  - Historical Source 原本は、再配布可能性を確認できたもの以外は Repository に格納しない。
  - GitHub 上の Issue / PR / Commit / Actions / Documentation / Historical Data を Single Source of Truth とする。
  - 最終 Web Application は **Desktop Browser + Smartphone Browser** を正式な利用対象とし、スマートフォン対応を後付け要件にしない。
  - SOL Independent Review は PR HEAD SHA に紐付け、HEAD 変更時には失効させる。
  - AI による自己承認・自己 Merge は禁止する。
  - 最終 Merge は Human Project Owner が実行する。
- 

## 1. 設計レビュー結果

前回案を、以下の追加前提に基づいて再レビューした。

1. GitHub は有料プランを使用しない。

- Human Project Owner はプログラミングを行わない。
- Human Project Owner はデバッグ・Terminal 操作を行わない。
- GitHub Cloud Only を維持する。
- AI が実装・検証・修正を担う。
- Historical Data の史料トレーサビリティを最優先する。

## 1.1 レビューで修正した主要点

| ID   | 論点                    | 旧案                  | 修正版                                         | 理由                                                          |
|------|-----------------------|---------------------|---------------------------------------------|-------------------------------------------------------------|
| R-01 | Repository owner      | Personal account 想定 | <b>GitHub Free Organization 推奨</b>          | Public repo で main への push 主体を限定しやすく、AI と Human の責任分離を強化できる |
| R-02 | Repository visibility | Private 開始案         | <b>Public</b>                               | GitHub Free で Protected Branch / Rulesets / Pages 等を活用するため  |
| R-03 | Codespaces            | 採用                  | <b>初期導入しない</b>                              | Human がコーディング・デバッグしないため常設価値が低い                              |
| R-04 | Human-only Merge      | 運用ルール中心             | <b>権限制御も併用</b>                              | AI が main を更新できる余地を可能な限り排除するため                              |
| R-05 | main protection       | Ruleset 中心          | <b>単一 Branch Protection Rule を初期推奨</b>      | 対象が main のみであり、より単純に必要な要件を満たせる                              |
| R-06 | PR Policy CI          | 通常 CI と同列           | <b>Trusted Policy Gate として分離</b>            | PR 自身による Policy Workflow 改変で Gate を自己無効化するリスクを下げる           |
| R-07 | Public repo contents  | 一般的な注意のみ            | <b>Public Repository Content Policy を追加</b> | Historical Source や Assets の再配布リスクを明示的に防ぐため                 |
| R-08 | Human 作業              | 手動 debug の余地あり      | <b>判断・承認・Merge に限定</b>                      | Project Owner にプログラミング技能を要求しないため                            |
| R-09 | Native approval       | 1 approval 候補       | <b>初期 0 approvals</b>                       | SOL Review は GitHub native human review とは別の独立レビューであるため     |
| R-10 | Production            | Pages 有力            | <b>後続決定</b>                                 | Technology Selection 前に static hosting 適合性を確定しないため          |
| R-11 | Client target         | Desktop 中心の暗黙前提     | <b>Desktop + Smartphone Browser を正式対象</b>   | 後続の MVP・UI・技術選定・テストでスマホを初期制約として扱うため                         |

## 2. Foundation との整合性

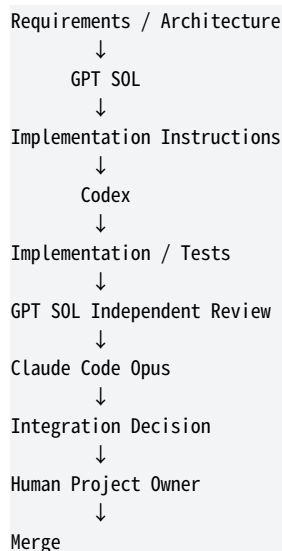
Project Foundation v0.1 の以下の原則は変更しない。

## 2.1 品質優先順位

史実性 > トレーサビリティ > 理解しやすさ > 網羅性 > 視覚的演出

GitHub 基盤はこの優先順位を支援するための開発インフラであり、Historical Data の内容そのものを決定するものではない。

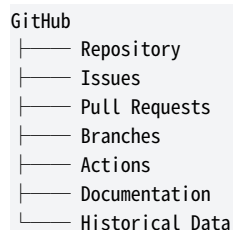
## 2.2 AI 責任分離



実装者と独立レビュー担当を分離する。

Codex による自己承認は禁止する。

## 2.3 Single Source of Truth

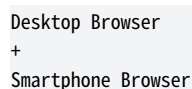


AI との会話ログは作業空間であり正本ではない。

重要な決定・設計・レビュー・変更履歴は GitHub へ反映する。

## 2.4 Target Client Constraint

最終成果物である Web Application は、以下を正式な利用対象とする。



スマートフォン対応は、Desktop 版完成後の追加対応ではなく、**MVP・UI 設計・Technology Selection・Test 設計に最初から影響する上位制約**として扱う。

ただし本 GitHub 基盤設計フェーズでは、以下をまだ確定しない。

- responsive layout の具体方式
- mobile-first / desktop-first の実装方式

- smartphone 向け 3D 描画品質
- 対応ブラウザの具体的 version matrix
- portrait / landscape の詳細仕様
- touch gesture 仕様
- device 別 performance budget

これらは後続フェーズで決定する。

本フェーズで固定するのは、\*\*「PC 専用の Web Application として設計してはならない」\*\*という制約である。

---

### 3. GitHub Cloud Only の運用定義

本プロジェクトにおける **GitHub Cloud Only** を以下のように定義する。

ソースコード、Historical Data、Issue、設計文書、変更履歴、Pull Request、テスト結果、Build、CI、レビュー記録、Deployment の正本および実行基盤を GitHub Cloud 上に置き、Human Project Owner のローカル開発環境を必要としない。

これは、すべての AI 推論処理そのものが GitHub サーバー上で動作しなければならない、という意味ではない。

AI が使用する実装環境は、GitHub Repository / Branch / PR と直接連携し、成果物を GitHub へ記録できるクラウド実行方式であることを要求する。

#### 3.1 Human Project Owner に要求しないもの

以下を Human Project Owner の責務に含めない。

- コーディング
- Terminal 操作
- Git CLI 操作
- Build コマンド実行
- Test コマンド実行
- Debugger 操作
- CI ログの技術解析
- Dependency 問題の技術修正
- Merge conflict の手動解消
- 開発環境構築

これらは AI または GitHub Actions が担当する。

---

### 4. Recommended GitHub Architecture

flowchart TD

```
H[Human Project Owner]
O[Claude Code Opus<br/>Overall Lead]
S[Claude Code Sonnet<br/>PMO]
```

```
G[GPT SOL<br/>Architecture / Independent Review]
C[Codex<br/>Implementation]

ORG[GitHub Free Organization]
I[Issues]
B[Task Branch]
P[Pull Request]
PG[Trusted PR Policy Gate]
CI[GitHub Actions CI]
R[SOL Independent Review]
M[Protected main]
D[Deployment<br/>Later Phase]
```

```
H --> ORG
ORG --> I
O --> I
S --> I
G --> I
I --> C
C --> B
B --> P
P --> PG
P --> CI
PG --> R
CI --> R
R --> O
O --> H
H --> M
M --> D
```

## 5. Account / Organization Design

### 5.1 推奨

**GitHub Free Organization** を作成し、その **Organization** 配下に **Public Repository** を 1 つ作成する。

#### 理由

Personal account 直下の Public Repository でも Protected Branch は利用できるが、本プロジェクトでは AI と Human Project Owner の責任分離が重要である。

GitHub Free Organization 配下の Public Repository では、main に対して push 可能な actor を限定できる構成が取りやすい。

これにより、

```
AI
└─ task branch / PR

Human Project Owner
└─ protected main への最終 Merge
```

という境界を GitHub 設定として表現できる。

### 5.2 Organization Role

Human Project Owner :

- Organization Owner
- Repository Admin

AI :

- Organization Owner にしない
- Repository Admin にしない
- Branch Protection bypass を与えない
- Secrets 管理権限を与えない
- Repository 削除権限を与えない

## 5.3 重要な制約

Human Project Owner と AI が同一 **GitHub Identity / 同一 Owner credential** を共有した場合、**Human-only Merge** を技術的に区別できない。

したがって AI integration には、可能な限り独立した GitHub App / service identity / 限定権限を使用する。

具体的な AI ごとの GitHub 接続方式は Repository 構築時に確認する。

---

## 6. Repository Design

### 6.1 Repository 数

#### 1 Repository

Multi-repository は採用しない。

理由 :

- Application と Historical Data の整合変更を同一 PR で扱える。
- Schema 変更と Data migration を atomic に検証できる。
- Source ID と Visualizer 実装の Traceability が単純になる。
- AI 間の Issue / PR 依存関係を増やさない。
- MVP 規模では分割メリットより運用コストが大きい。

### 6.2 Visibility

#### Public

GitHub Free で main protection、Actions、Pages 等を最大限活用するため。

#### 注意

Public Repository であることと、Open Source License を付与することは別である。

**LICENSE は自動的に追加しない。**

ライセンス選定は、コード・Historical Dataset・Assets の公開方針を確認後に決定する。

---

## 7. Repository Directory Baseline

```
/
├── .github/
│   ├── ISSUE_TEMPLATE/
│   │   ├── general.yml
│   │   └── historical.yml
│   ├── workflows/
│   │   ├── policy-gate.yml
│   │   └── repository-validation.yml
│   └── pull_request_template.md
├── docs/
│   ├── foundation/
│   ├── architecture/
│   ├── governance/
│   └── decisions/
├── src/
├── tests/
├── assets/
├── historical/
│   ├── data/
│   ├── sources/
│   ├── schemas/
│   └── validation/
├── scripts/
└── README.md
```

### 7.1 未確定領域

以下は namespace のみ確保し、詳細設計を先行しない。

- historical/data/
- historical/sources/
- historical/schemas/
- historical/validation/

具体的 Schema、Source ID 形式、Confirmed / Estimated / Unknown 等は Historical Data Model フェーズで決定する。

---

## 8. Public Repository Content Policy

Public Repository に commit する情報は、公開可能であることを前提とする。

### 8.1 Repository に格納可能

- Source Code
- Tests
- Documentation
- Historical Dataset
- Historical Source Metadata

- Schema
- Validation rules
- 自作 Assets
- Public Domain Assets
- 再配布許諾が確認できた Assets

## 8.2 原則格納禁止

- 購入した電子書籍
- 書籍 PDF
- 無許可のスキャン史料
- 再配布条件不明の地図
- 再配布条件不明の写真
- ライセンス不明 Assets
- API Key
- Token
- Password
- Secret
- 個人情報
- 契約上非公開の情報

## 8.3 Historical Source

Source 原本を Repository に置くことを Traceability の要件とはしない。

最低限、

```
Source ID
Title
Author / Institution
Publication
Page
URL / Archive ID
Access Metadata
```

等を通じて元史料へ逆引き可能な構造を作る。

原本ファイルを格納する場合のみ、再配布可能性を確認する。

---

# 9. Branch Strategy

## 9.1 Branch 種類

```
main
|— feature/<issue>-<slug>
|— fix/<issue>-<slug>
|— data/<issue>-<slug>
|— docs/<issue>-<slug>
|— infra/<issue>-<slug>
```



## 9.2 原則

- main は常に統合済み正本。
- task branch は最新 main から作る。
- 原則として 1 Issue = 1 branch = 1 PR。
- branch は短命とする。
- merge 後に自動削除する。
- develop は作らない。
- release/\* は初期導入しない。
- hotfix/\* 専用フローは作らない。
- 長期統合 branch は作らない。

## 9.3 理由

AI 開発では branch 種類を増やすほど、

- 正本判断
- merge 順序
- dependency 管理
- review 対象 SHA

が複雑化する。

本プロジェクトでは main 中心の最小構成を採用する。

---

## 10. Merge Strategy

Repository 設定：

```
Squash Merge: ON  
Merge Commit: OFF  
Rebase Merge: OFF  
Auto Merge: OFF  
Automatically delete head branches: ON
```

### 10.1 Squash 採用理由

- 1 PR = 1 logical change として main 履歴を残せる。
  - AI 実装中の細かい修正 commit を main へ持ち込まない。
  - Issue / PR / main commit の対応関係を追やすい。
  - revert 単位が明確になる。
-

## 11. main Branch Protection

初期構成では、複数 Ruleset を組み合わせず、**main 専用の Branch Protection Rule** を 1 つ使用する。

### 11.1 main protection 設定

| Setting                                    | 初期値                    |
|--------------------------------------------|------------------------|
| Require a pull request before merging      | ON                     |
| Required approving reviews                 | 0                      |
| Require status checks before merging       | ON                     |
| Require branches to be up to date          | ON                     |
| Require conversation resolution            | ON                     |
| Restrict who can push to matching branches | Human Project Owner のみ |
| Allow force pushes                         | OFF                    |
| Allow deletions                            | OFF                    |
| Do not allow bypassing the above settings  | ON                     |
| Require signed commits                     | OFF                    |
| Require linear history                     | OFF                    |
| Require merge queue                        | OFF                    |
| Require deployments before merging         | OFF                    |

### 11.2 Required approving reviews を 0 とする理由

GitHub native approval と SOL Independent Review は役割が異なる。

SOL が独立 GitHub reviewer identity を持つことは現時点では確定していない。

また Human Project Owner 自身の PR に対して Human 本人の approval を要求する設計は責任分離として意味を持たない。

そのため、

Native GitHub approval  
≠  
SOL Independent Review

とする。

SOL Review は別途 Policy Gate で扱う。

### 11.3 Human-only Merge

Protected main への push 可能 actor を Human Project Owner に限定する。

AI user / AI GitHub App を main push 許可対象へ追加しない。

AI には admin / bypass 権限を与えない。

これにより、AI は task branch と PR までを担当し、main 更新は Human Project Owner へ残す。

## Identity 制約

この技術的分離は、AI が Human Project Owner とは異なる GitHub identity を使用する場合に成立する。

AI が Project Owner の owner session そのものを使用する場合は完全な技術分離ができない。

そのため Repository 構築時に AI integration 方式を確認し、Owner credential を AI へ恒常的に渡さない。

---

## 12. Pull Request Workflow

```
Issue
↓
Opus: Priority / Go
↓
Sonnet: Issue readiness / dependencies
↓
Codex: task branch 作成
↓
Codex: Implementation + Tests
↓
Draft PR
↓
Repository Validation
↓
Application CI
↓
Ready for Review
↓
PR Policy Gate
↓
SQL Independent Review
↓
PASS / CHANGES_REQUIRED
↓
[変更があれば]
Codex 修正
↓
HEAD SHA 変更
↓
SQL Review 失効
↓
再 CI
↓
SQL 再 Review
↓
Opus Integration Decision
↓
Human Project Owner
↓
Squash Merge
```

---

## 13. Pull Request Template

```
## Objective

## Related Issue

## Scope

## Changes
```

```

## Tests

## Historical Data Impact
NONE / YES

## Source Traceability Impact
NONE / YES

## Client Compatibility Impact
NONE / DESKTOP / SMARTPHONE / BOTH

## Known Limitations

## Current HEAD

---

## SOL Independent Review

SOL review commit:
SOL review decision:
BLOCKER:
MUST FIX:
Current HEAD:

---

## Integration Decision

Opus decision:
Decision HEAD:

```

Historical Data Impact が YES の場合：

```

## Historical Change Detail

Affected Source IDs:
Source / Evidence:
Traceability Impact:
Validation Result:
Known Uncertainty:

```

---

## 14. SOL Independent Review

### 14.1 Review identity

SOL Review は**特定 PR HEAD SHA に対する独立レビュー**とする。

PASS 条件：

```

SOL review commit == actual PR HEAD SHA
Current HEAD == actual PR HEAD SHA
SOL review decision == PASS
BLOCKER == 0
MUST FIX == 0

```

### 14.2 HEAD 変更

SOL PASS 後に新 commit が push された場合：

```

Reviewed HEAD = abc123
New HEAD      = def456

```

となるため、

```
SOL REVIEW = STALE / INVALID
```

と判定する。

再レビューを完了するまで Merge 不可。

## 14.3 Identity の限界

PR 本文だけでは、

「SOL review decision: PASS」

を書いた actor が本当に SOL であることまで完全には証明できない。

したがって初期構成では、

1. SHA 一致
2. Policy Gate
3. SOL 独立レビュー
4. Opus 統括判断
5. Human-only Merge

を組み合わせる。

将来、SOL 専用 GitHub App / reviewer identity を用意できた場合は、review actor identity validation を追加検討する。

---

## 15. CI/CD Foundation

CI は段階導入する。

Visualization 技術や package manager を現段階で仮定しない。

### 15.1 Phase 0 — GitHub Foundation

初期導入：

#### A. policy-gate.yml

役割：

- PR 本文必須欄確認
- Related Issue 確認
- Current HEAD 整合確認
- SOL review commit 整合確認
- PASS 時 BLOCKER = 0 確認
- PASS 時 MUST FIX = 0 確認
- Historical Data Impact 欄確認

- Source Traceability Impact 欄確認

## B. repository-validation.yml

役割：

- Repository 基本構造
- 必須文書
- 基本 format
- Foundation / Governance 整合性チェック

Technology Selection 前のため、特定言語の build/test は要求しない。

---

## 16. Trusted PR Policy Gate

PR Policy Gate は通常の Application CI と分離する。

### 16.1 問題

通常の pull\_request workflow だけを使用すると、PR 自身が Policy Workflow を変更するケースを考慮する必要がある。

Governance Gate が PR 側の変更に直接依存すると、実装者が Gate 自体を変更できる構造になり得る。

### 16.2 設計

Policy Gate は **base branch 側の信頼済み Workflow 定義を使用する構成**とする。

候補として pull\_request\_target を使用できる。

ただし、pull\_request\_target は権限の強い context になり得るため、以下を厳守する。

#### MUST

- PR code を checkout しない。
- PR branch 上の script を実行しない。
- PR 由来の実行可能コードを呼ばない。
- Secret を渡さない。
- permissions を明示的な最小 read 権限とする。
- GitHub event metadata / API 上の PR metadata だけを検証する。
- PR title / body 等を shell command へ未処理で埋め込まない。

つまり Policy Gate は、

```
PR Metadata
PR HEAD SHA
Issue relation
Review fields
```

だけを見る。

Application code の Build / Test は通常の pull\_request CI で実行する。

## 16.3 分離

```
Trusted Policy Gate
└─ Governance / metadata のみ

Application CI
└─ PR code / tests / build
```

この 2 つを混ぜない。

## 16.4 Governance-sensitive paths

以下は通常の Feature / Data 変更より強く扱う。

```
.github/workflows/**
.github/ISSUE_TEMPLATE/**
.github/pull_request_template.md
docs/foundation/**
docs/governance/**
```

原則：

- infra/\* または明示的な governance 変更 PR として分離する。
- 通常 Feature PR へ便乗させない。
- 変更理由と影響範囲を PR 本文へ明記する。
- SOL Independent Review を必須とする。
- Opus Integration Decision を必須とする。
- Human Project Owner が最終 Merge する。
- policy-gate.yml の変更は、その PR 自身の Policy Gate 判定ロジックへ即時反映させない。base branch 上の信頼済み定義で当該 PR を評価し、Human Merge 後に将来の PR へ反映する。

これにより、実装 AI が通常の機能変更に分れてガバナンスや CI Gate 自体を弱めることを防ぐ。

---

## 17. Application CI

Technology Selection 後に追加する。

Desktop / Smartphone の双方を正式な利用対象とするため、後続の CI / Test 設計では、少なくとも以下を検討対象に含める。

- responsive layout regression
- smartphone viewport での主要画面確認
- touch 操作を前提とした主要 interaction
- mobile browser での build/runtime compatibility
- device 性能差を考慮した performance validation

ただし具体的な browser matrix、E2E tool、viewport 値、performance threshold は Technology Selection 後に決定する。

基本構造：

```
Install
↓
Lint
↓
Type Check
↓
Unit Test
↓
Build
↓
CI / required
```

Ruleset / Branch Protection から参照する Required Check 名は安定させる。

例：

```
policy / pr-policy
ci / required
```

Workflow 内部 job を変更しても、外部 Required Check 名を不用意に変えない。

---

## 18. Historical Data Validation

Historical Data Model 確定後に導入する。

候補：

- Schema Validation
- Source ID Validation
- Source Traceability Validation
- Broken Reference Validation
- Duplicate Source ID Validation
- Orphaned Historical Data Validation
- Required Evidence Validation
- Historical numeric change validation
- Source deletion / rename detection

### 18.1 Historical Data 変更ルール

historical/\*\* を変更する PR は必ず、

```
Historical Data Impact = YES
Source / Evidence
Affected Source IDs
Traceability Impact
```

を要求する。

### 18.2 Source ID

Source ID は将来的に恒久 identifier として扱う方向を推奨する。

ただし形式・immutability の詳細は Historical Data Model / Source Acceptance Policy フェーズで正式決定する。

---



## 19. Issue Management

Issue Template は過剰分割せず 2 系統とする。

### 19.1 General Work Item

対象：

- Feature
- Bug
- Design
- Infrastructure
- Documentation

必須欄：

Objective  
Scope  
Requirements  
Constraints  
Acceptance Criteria  
Dependencies  
Source / Evidence  
Out of Scope

### 19.2 Historical Work Item

対象：

- Historical Data
- Research

追加必須欄：

Historical Objective  
Target Period  
Target Geography  
Target Units / Events  
Source / Evidence  
Source Tier  
Known Uncertainty  
Traceability Impact

### 19.3 Stale Issue / PR

自動 stale-close は導入しない。

歴史調査は長期間停止していても無効になったとは限らない。

Sonnet PMO が状態を確認し、

- Active
- Blocked
- Superseded
- Closed

を判断可能な状態へ整理する。

Close 判断は必要に応じて Opus へエスカレーションする。

---

## 20. AI Role & Permission Matrix

### 20.1 Responsibility

| 操作                   | Opus | Sonnet | SOL      | Codex | Human |
|----------------------|------|--------|----------|-------|-------|
| 全体方針                 | ◎    | △      | 設計助言     | ×     | 最終    |
| Issue 起票             | ○    | ◎      | 提案       | △     | ○     |
| Issue 管理             | ○    | ◎      | △        | ×     | ○     |
| branch 作成            | ×    | △      | ×        | ◎     | ○     |
| 実装                   | ×    | ×      | ×        | ◎     | ×     |
| Test 実装              | ×    | ×      | ×        | ◎     | ×     |
| PR 作成                | ×    | △      | ×        | ◎     | ○     |
| PR metadata 管理       | △    | ◎      | Review 欄 | 自 PR  | ○     |
| CI 確認                | ○    | ◎      | Review 時 | 修正時   | 結果確認  |
| Independent Review   | ×    | ×      | ◎        | 禁止    | △     |
| Integration 判断       | ◎    | △      | 助言       | ×     | 最終    |
| Merge                | ×    | ×      | ×        | ×     | ◎     |
| Branch Protection 変更 | ×    | ×      | ×        | ×     | ◎     |
| Secrets 管理           | ×    | ×      | ×        | ×     | ◎     |

### 20.2 Desired GitHub Permissions

#### Human Project Owner

- Organization Owner
- Repository Admin
- Merge
- Branch protection settings
- Secrets settings
- Repository settings

#### Claude Code Opus

原則：

- Contents: Read
- Issues: Read / Write as needed
- Pull Requests: Read / comment/update as needed
- Actions: Read
- Administration: None

- Secrets: None
- Merge: None

## Claude Code Sonnet

原則：

- Contents: Read
- Issues: Read / Write
- Pull Requests: Read / Write metadata
- Actions: Read
- Administration: None
- Secrets: None
- Merge: None

## GPT SOL

原則：

- Contents: Read
- Issues: Read
- Pull Requests: Read
- Actions: Read
- Review record write: only if supported by dedicated integration
- Contents Write: None
- Administration: None
- Secrets: None
- Merge: None

## Codex

必要範囲：

- Contents: task branch への Write
- Pull Requests: Read / Write
- Issues: Read
- Actions: Read
- Administration: None
- Secrets: None
- main push permission: None
- Merge: None

具体的 scope 名称は、利用する GitHub App / AI integration の仕様確認後にマッピングする。

---

## 21. Codex Development Strategy

Human Project Owner は開発環境を操作しない。

Implementation agent である Codex は、

1. GitHub Issue を読む。
2. task branch を作成する。
3. 実装する。
4. Test を作成する。
5. branch へ commit / push する。
6. PR を作成する。
7. GitHub Actions 結果を確認する。
8. 失敗時に修正する。
9. PASS 状態を SOL へ引き渡す。

というクラウドベース運用を要求する。

### 21.1 Test の正本

Codex 内部で実施したテスト結果のみでは Merge 条件を満たさない。

**GitHub Actions 上で再現された PASS を正本とする。**

```
graph TD; A[Codex local/internal test] --> B[参考]; B --> C[GitHub Actions]; C --> D[Authoritative CI Result]
```

## 22. Codespaces Strategy

### Decision

**INITIAL: NOT ADOPTED**

Codespaces は初期必須構成から外す。

### 理由

Human Project Owner が以下を行わないため。

- coding
- interactive debugging
- Terminal
- manual build
- environment setup

また、Build / Test / Validation は GitHub Actions で実行する。

## 将来導入条件

以下の問題が実際に発生した場合だけ再検討する。

- AI 実装環境と GitHub Actions の再現差異が重大化した。
- 3D 描画の対話的デバッグ環境が必須になった。
- GitHub Actions ログだけでは障害再現が困難になった。
- 共通 devcontainer が開発効率に明確な利益をもたらす。

その場合も、Codespaces を Human Project Owner が操作することは前提にしない。

---

## 23. Security Baseline

### 23.1 Authentication

- Human Project Owner は 2FA を使用する。
- Human Owner credential を AI へ恒常共有しない。
- AI は可能な限り独立 GitHub App / 限定 identity を利用する。
- PAT が必要な場合は fine-grained かつ最小 scope とする。
- classic PAT は原則使用しない。

### 23.2 GitHub Actions

Workflow ごとに permissions を明示する。

原則：

```
permissions:  
  contents: read
```

追加権限は必要な Workflow にのみ付与する。

### 23.3 Third-party Actions

外部 Action を使用する場合は、

- 必要性を確認する。
- GitHub 公式 Action を優先する。
- 第三者 Action は full commit SHA pinning を原則とする。
- 不要な Action を増やさない。

### 23.4 Secrets

- Secret を source へ commit しない。
- PR CI へ Secrets を不用意に渡さない。
- Public Repositoryであることを前提に扱う。

- Secret が不要な設計を優先する。

## 23.5 Security Features

Public Repository で無料利用可能な GitHub security 機能は可能な範囲で有効化する。

候補：

- Dependabot Alerts
- Secret Scanning
- Code Scanning
- Dependency Review

ただし package manager 等に依存する設定は Technology Selection 後に追加する。

---

## 24. Historical Data Protection Strategy

Historical Data は通常コードより強い変更規則を持つ。

### 24.1 防止対象

- 根拠なしデータ追加
- Source ID 破壊
- Source 削除
- Source 参照切断
- 数値だけの上書き
- 出典情報の欠落
- Schema 不整合
- 史料上確認できない値の Confirmed 扱い

### 24.2 GitHub 上の防護層

```
graph TD
    A[Historical Work Item] --> B[data/* branch]
    B --> C[PR Historical Data Impact]
    C --> D[Historical Validation]
    D --> E[SOL Independent Review]
    E --> F[Opus Integration Decision]
    F --> G[Human Merge]
```

### 24.3 Code と Data の Repository 分離

初期段階では分離しない。

理由：

Schema 変更  
+  
Validator 変更  
+  
Historical Data migration

を同一 PR で atomic に検証できることの価値が高いため。

将来 Repository size、license、権限分離等に明確な問題が発生した場合のみ再評価する。

---

## 25. Deployment / Preview Strategy

### 25.1 Production

GitHub Pages を第一候補とする。

ただし正式採用は Technology Selection 後。

条件：

Application build  
↓  
Static browser assets として公開可能

であること。

### 25.2 PR Preview

初期導入しない。

理由：

- Visualization 技術未選定
- 外部 SaaS を増やさない
- MVP 前に必要性が未確認
- CI artifacts だけで足りる可能性がある

3D 画面レビューが高頻度化し、URL ベース PR Preview の利益が明確になった場合だけ再設計する。

### 25.3 Development Preview

Human Project Owner 向け開発環境としては用意しない。

AI が必要とする一時的 Preview 方法は Technology Selection 後に実装方式と合わせて検討する。

---

## 26. GitHub Artifacts / Releases

### GitHub Actions Artifacts

必要時採用

用途：

- Build output
- Test report
- Validation report

Historical Source archive には使用しない。

## GitHub Releases

初期導入しない。

以下の milestone 発生後に採用検討する。

- MVP v0.1
  - Public dataset baseline
  - Stable release
- 

## 27. Initial Repository Setup Sequence

実際の構築フェーズでは以下の順序で行う。

1. GitHub Free Organization 作成
2. Human Project Owner を Organization Owner とする
3. Public Repository 作成
4. Default branch=main
5. Squash Merge のみ有効化
6. Merge Commit 無効化
7. Rebase Merge 無効化
8. Auto Merge 無効化
9. Automatically delete head branches 有効化
10. Project Foundation v0.1 登録
11. 本 GitHub Cloud Development Foundation 登録
12. docs/基本構造作成
13. .github/基本構造作成
14. Issue Templates 作成
15. Pull Request Template 作成
16. Trusted Policy Gate 作成
17. Repository Validation 作成
18. main Branch Protection 作成
19. PR 必須化
20. Required Status Check 設定



21. Conversation Resolution 必須化
  22. Force Push 禁止確認
  23. main deletion 禁止確認
  24. Branch Protection bypass 禁止
  25. main push actor を Human Project Owner に限定
  26. AI identities / GitHub integration 方式確認
  27. AI に Admin / Owner 権限がないことを確認
  28. AI に main push / Merge 能力がないことを確認
  29. Actions token 権限最小化
  30. Security 機能有効化
  31. Public Repository Content Policy 確認
  32. Historical namespace のみ作成
  33. テスト用 PR 作成
  34. CI 失敗時に Merge できないことを確認
  35. SOL review SHA 不一致時に Policy Gate が失敗することを確認
  36. Human Project Owner 以外が main を更新できないことを確認
  37. Merge 後 branch 自動削除確認
  38. Opus が基盤完成状態を確認
  39. GitHub 開発基盤フェーズ完了を GitHub へ記録
  40. 次フェーズへ移行
- 

## 28. Human Project Owner Operating Model

Project Owner が通常行う GitHub 操作は最小限とする。

### 日常

- Issue / PR 状況確認
- AI からの判断要求への回答
- SOL review 結果確認
- Opus integration decision 確認

### Merge 時

確認対象：

```
Required Checks = PASS
SOL Decision = PASS
SOL Review HEAD = Current PR HEAD
BLOCKER = 0
MUST FIX = 0
Opus Integration Decision = APPROVE
```

上記を満たす場合に Human Project Owner が Squash Merge を実行する。

Human Project Owner がコードを読んで技術的正当性を自力で判定することを Merge 条件とはしない。

技術的正当性は、

```
Codex Tests
+
GitHub Actions
+
SQL Independent Review
```

によって担保する。

---

## 29. Open Decisions

以下は本フェーズで確定しない。

| 項目                                             | 決定時期                                  |
|------------------------------------------------|---------------------------------------|
| Organization 名                                 | Repository Setup                      |
| Repository 名                                   | Repository Setup                      |
| 各 AI の具体的 GitHub identity / App                | Repository Setup                      |
| LICENSE                                        | 公開・権利方針確定後                            |
| Frontend language                              | Technology Selection                  |
| Framework                                      | Technology Selection                  |
| 3D Library                                     | Technology Selection                  |
| Package Manager                                | Technology Selection                  |
| Historical Schema                              | Historical Data Model                 |
| Source ID 形式                                   | Historical Data Model                 |
| Confirmed / Estimated / Unknown                | Historical Data Model                 |
| Source Acceptance 詳細基準                         | Source Policy                         |
| Historical Validators 詳細                       | Data Model 後                          |
| GitHub Pages 正式採用                              | Technology Selection 後                |
| PR Preview                                     | Visualization 実装後                     |
| 対応 Desktop / Smartphone browser matrix         | Technology Selection / MVP UI 設計      |
| Smartphone viewport / orientation / touch 詳細仕様 | MVP UI 設計                             |
| Mobile performance budget                      | Technology Selection / Performance 設計 |
| GitHub Releases                                | MVP milestone 前                       |

## 30. Explicit Non-Adoption

現時点では以下を採用しない。

- Private Repository
- GitHub 有料プラン
- Multi Repository
- develop branch

- Git Flow
  - Merge Queue
  - Auto Merge
  - AI による Merge
  - Codespaces 常設
  - github.dev を Human 開発環境とする構成
  - Human による手動 debug
  - Human による Terminal 操作
  - Microservices
  - GitHub Projects 必須化
  - 外部有料 Cloud
  - 外部 Preview SaaS
  - Codespaces Prebuild
  - GitHub Packages
  - Historical Schema 先行設計
  - Visualization 技術先行選定
  - 自動 stale-close
- 

## 31. Design Change Conditions

現在の結論を変更する条件を明示する。

### Public → Private

以下のいずれかが発生した場合：

- 公開できないコードが必要になる。
- Historical Data 自体に非公開要件が発生する。
- License / source 条件により公開 Repository が不適切になる。
- GitHub 有料プラン使用を許容する方針へ変更される。

### Single Repository → Multi Repository

以下が実証された場合のみ：

- Repository size が実運用上問題になる。
- Historical source licensing 上、物理分離が必要になる。
- 明確に異なる access control が必要になる。
- CI / deployment independence の利益が運用コストを上回る。

## Codespaces 導入

以下が発生した場合：

- Actions だけでは再現不能なデバッグ問題が継続する。
- AI 間の開発環境差異が重大な問題になる。
- interactive 3D debugging が必須になる。

## GitHub Pages 不採用

Technology Selection 後、static hosting 要件を満たせない場合。

---

## 32. Self Review

BLOCKER:

0

MUST FIX:

0

SHOULD FIX:

1

Repository Setup 開始前に、  
各 AI が GitHub 上での identity / GitHub App / credential を使用するか確認し、  
Human Project Owner の Owner/Admin identity と分離できることを確認する。

NICE TO HAVE:

3

1. SOL 専用 GitHub review identity
2. Historical Validation report の可視化
3. Visualization 実装後の PR Preview

FINAL DECISION:

PASS

---

## 33. Final Architecture Decision

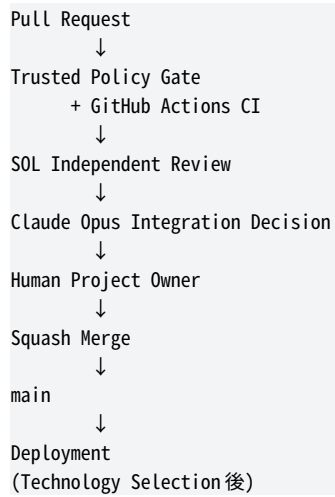
本設計を、Project Foundation v0.1 を変更せず具体化する

### GitHub Cloud Development Foundation v0.3

として採用候補とする。

最終構成：

```
GitHub Free Organization
  ↓
Public Single Repository
  ↓
Protected main
  ↓
Issue
  ↓
AI task branch
  ↓
```



本構成の目的は、

Human Project Owner にプログラミング技能を要求せず、AI による設計・実装・検証・独立レビューを GitHub 上で追跡可能にし、Historical Data の史料トレーサビリティを破壊せず、Desktop / Smartphone の双方を正式な出力先として、安全かつ再現可能な開発を継続できる状態を作ること

である。